

Android XML 原生TimePickerDialog详解（基础+自定义）

在 Android 开发中，`TimePickerDialog` 是系统封装的「时间选择对话框」，专为获取用户输入的时间（时/分）设计，广泛应用于闹钟设置、预约时间配置、日程安排等场景。它继承自 `AlertDialog`，内置 `TimePicker` 控件，支持12小时制与24小时制切换，兼具原生适配性和扩展灵活性。本文从「原生基础用法」到「深度自定义开发」全面拆解，搭配可直接运行的代码示例，帮助开发者快速掌握其使用技巧与定制方法。

一、核心概念：TimePickerDialog 本质与特性

`TimePickerDialog` 并非独立控件，而是「TimePicker + AlertDialog」的组合封装，核心特性如下：

- 模态交互**：弹出后阻断当前页面操作，需用户完成时间选择或取消操作才能关闭，确保时间输入的完整性
- 双时间制支持**：默认跟随系统时间制，可手动指定12小时制（含上下午标识）或24小时制，适配不同地区使用习惯
- 原生样式适配**：样式与系统主题自动同步，高版本（API 21+）支持通过主题属性快速调整颜色、字体等视觉元素
- 简洁回调机制**：通过 `OnTimeSetListener` 直接返回用户选择的时间，无需手动从 `TimePicker` 中获取数据
- 轻量扩展**：支持自定义按钮文本、对话框标题，也可通过替换 `TimePicker` 实现复杂视觉定制

 **关键区别**：`TimePicker` 是单独的时间选择控件，可嵌入布局使用；`TimePickerDialog` 是封装好的对话框，无需额外处理弹窗逻辑，适合快速实现时间选择功能。

二、基础用法：原生 TimePickerDialog（3步快速实现）

原生 `TimePickerDialog` 采用 Builder 模式构建，核心流程为「初始化对话框配置 → 设置时间选择回调 → 显示对话框」，适合简单的时间选择场景。

步骤 1：布局准备（触发按钮与结果显示）

在 `activity\_main.xml` 中添加触发对话框的按钮和显示选择结果的文本控件：

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:orientation="vertical"
7     android:padding="20dp"
8     android:gravity="center"
9     android:background="@color/gray_f5">
10
11
12     <Button
13         android:id="@+id/btn_show_time_picker"
14         android:layout_width="wrap_content"
15         android:layout_height="wrap_content"
16         android:text="选择预约时间"
17         android:textSize="16sp"
18         android:backgroundTint="@color/blue"/>
19
20
21     <TextView
22         android:id="@+id/tv_selected_time"
23         android:layout_width="wrap_content"
24         android:layout_height="wrap_content"
25         android:layout_marginTop="20dp"
26         android:text="已选择时间：暂无"
27         android:textSize="16sp"
28         android:textColor="@color/black"/>
29
30
31     <TextView
32         android:id="@+id/tv_formatted_time"
33         android:layout_width="wrap_content"
34         android:layout_height="wrap_content"
35         android:layout_marginTop="10dp"
36         android:text="接口格式：暂无"
37         android:textSize="14sp"
38         android:textColor="@color/gray_99"/>
39
40 </LinearLayout>

```

补充颜色资源（res/values/color.xml）：

Code block

```

1 <resources>
2     <color name="white">#FFFFFF</color>

```

```

3      <color name="gray_f5">#F5F5F5</color>
4      <color name="gray_e0">#E0E0E0</color>
5      <color name="gray_99">#999999</color>
6      <color name="black">#000000</color>
7      <color name="blue">#2196F3</color>
8      <color name="blue_dark">#1976D2</color>
9      <color name="purple_500">#9C27B0</color>
10     <color name="red">#F44336</color>
11 </resources>

```

步骤 2：在 Activity 中实现对话框逻辑

在 `MainActivity.kt` 中绑定控件，初始化 `TimePickerDialog` 并设置时间选择回调：

Code block

```

1  import android.os.Bundle
2  import android.widget.Button
3  import android.widget.TextView
4  import android.widget.TimePicker
5  import android.widget.Toast
6  import androidx.appcompat.app.AppCompatActivity
7  import androidx.appcompat.app.TimePickerDialog
8  import java.text.SimpleDateFormat
9  import java.util.Calendar
10 import java.util.Locale
11
12 class MainActivity : AppCompatActivity() {
13     private lateinit var tvSelectedTime: TextView
14     private lateinit var tvFormattedTime: TextView
15
16     override fun onCreate(savedInstanceState: Bundle?) {
17         super.onCreate(savedInstanceState)
18         setContentView(R.layout.activity_main)
19
20         // 绑定控件
21         tvSelectedTime = findViewById(R.id.tv_selected_time)
22         tvFormattedTime = findViewById(R.id.tv_formatted_time)
23         val btnShowTimePicker = findViewById<Button>(R.id.btn_show_time_picker)
24
25         // 按钮点击触发对话框
26         btnShowTimePicker.setOnClickListener {
27             showNativeTimePickerDialog()
28         }
29     }
30
31     // 显示原生 TimePickerDialog

```

```

32     private fun showNativeTimePickerDialog() {
33         // 1. 获取当前时间（作为默认选中时间）
34         val calendar = Calendar.getInstance()
35         val currentHour = calendar.get(Calendar.HOUR_OF_DAY) // 24小时制的小时
36         val currentMinute = calendar.get(Calendar.MINUTE) // 分钟
37
38         // 2. 创建 TimePickerDialog 实例
39         val timePickerDialog = TimePickerDialog(
40             this, // 上下文
41             { _: TimePicker, selectedHour: Int, selectedMinute: Int ->
42                 // 3. 时间选择回调（用户点击"确定"后触发）
43                 handleSelectedTime(selectedHour, selectedMinute)
44             },
45             currentHour, // 默认选中的小时（24小时制）
46             currentMinute, // 默认选中的分钟
47             true // 是否使用24小时制（true=24小时制，false=12小时制）
48         )
49
50         // 4. 可选配置：自定义对话框样式与行为
51         timePickerDialog.apply {
52             // 设置对话框标题
53             setTitle("选择预约时间（仅工作时间）")
54             // 设置是否可取消（点击外部或返回键关闭）
55             setCancelable(true)
56             // 自定义取消按钮文本与点击事件
57             setButton(TimePickerDialog.BUTTON_NEGATIVE, "取消") { dialog, _ ->
58                 dialog.dismiss()
59                 Toast.makeText(this@MainActivity, "已取消时间选择",
60                     Toast.LENGTH_SHORT).show()
61             }
62             // 自定义确认按钮文本（默认是"确定"）
63             setButton(TimePickerDialog.BUTTON_POSITIVE, "完成") { dialog, _ ->
64                 // 此处无需手动处理时间，回调会自动触发，仅用于扩展逻辑
65                 dialog.dismiss()
66             }
67         }
68
69         // 5. 显示对话框
70         timePickerDialog.show()
71     }
72
73     // 处理选中的时间（格式化与显示）
74     private fun handleSelectedTime(hour: Int, minute: Int) {
75         // 1. 校验时间合法性（示例：仅允许选择 9:00-18:00 的工作时间）
76         if (hour < 9 || hour > 18) {
77             Toast.makeText(this, "请选择9:00-18:00的工作时间",
78                 Toast.LENGTH_SHORT).show()
79         }
80     }

```

```
77         return
78     }
79
80     // 2. 显示带格式的时间 (如 14:05 或 下午2:05)
81     val timeStr = if (hour <= 12) {
82         val period = if (hour == 0) "凌晨" else if (hour == 12) "中午" else
            "上午"
83         val displayHour = if (hour == 0) 12 else hour
84         "$period $displayHour:${String.format(Locale.CHINA, "%02d",
            minute)}"
85     } else {
86         val period = if (hour == 12) "中午" else "下午"
87         val displayHour = hour - 12
88         "$period $displayHour:${String.format(Locale.CHINA, "%02d",
            minute)}"
89     }
90     return timeStr
91 }
```

步骤 3：原生 TimePickerDialog 核心 API 解析

TimePickerDialog 的核心配置通过构造函数和成员方法实现，以下是高频 API 说明：

1. 构造函数核心参数

参数名	类型	作用	注意事项
context	Context	关联的上下文，通常为 Activity	避免使用 Application，可能导致样式问题
listener	TimePickerDialog.OnTimeSetListener	时间选择回调，返回选中的小时和分	回调中的 hour 与时间制一致（24/12 小时制）
hourOfDay	Int	默认选中的小时（24 小时制）	若构造函数最后传入 false（12 小时制），仍需传入 24 小时制的小时数
minute	Int	默认选中的分钟	范围为 0-59，超出范围会取模
is24HourView	Boolean	是否使用 24 小时制	true=24 小时制（含 AM/PM），false=12 小时制（含上午/下午）

2. 常用成员方法

方法名	作用	示例
setTitle(CharSequence)	设置对话框标题	setTitle("选择闹钟时间")
setCancelable(Boolean)	设置是否可取消	setCancelable(false)（必须选择时间才能关闭）

setButton(int, String, DialogInterface.OnClickListener)	自定义按钮文本与点击事件	setButton(BUTTON_NEGATIVE, "取消", { dialog, _ -> dialog.dismiss() })
getTimePicker()	获取对话框中的 TimePicker 控件	val timePicker = dialog.getTimePicker()（用于进一步配置）

3. TimePicker 核心属性（通过 getTimePicker() 获取后配置）

属性名	作用	示例
hour	获取/设置选中的小时（与时间制一致）	timePicker.hour = 14（24小时制）
minute	获取/设置选中的分钟	timePicker.minute = 30
is24HourView	获取/设置时间制	timePicker.is24HourView = true
setOnTimeChangeListener	监听时间实时变化（滚轮滚动时触发）	timePicker.setOnTimeChangeListener { _, hour, minute -> ... }

三、进阶：自定义 TimePickerDialog 样式（视觉定制）

原生 `TimePickerDialog` 的样式受系统主题限制，若需适配 App 视觉风格（如圆角、自定义颜色），可通过「自定义主题」或「替换布局」两种方案实现。

场景 1：通过主题修改颜色与字体

核心思路：在 `styles.xml` 中定义自定义主题，修改 `TimePicker` 和对话框的视觉属性，再通过 `TimePickerDialog` 构造函数传入主题 ID。

步骤 1：定义自定义主题（res/values/styles.xml）

Code block

```
1  <resources>
2
3      <style name="AppTheme" parent="Theme.AppCompat.Light.DarkActionBar">
4          <item name="colorPrimary">@color/blue</item>
5          <item name="colorPrimaryDark">@color/blue_dark</item>
6          <item name="colorAccent">@color/blue</item>
7      </style>
```

```

8
9
10     <style name="CustomTimePickerTheme"
parent="Theme.AppCompat.Light.Dialog.Alert">
11
12         <item name="android:windowBackground">@drawable/dialog_bg</item>
13
14         <item name="android:windowTitleStyle">@style/DialogTitleStyle</item>
15
16         <item name="colorAccent">@color/purple_500</item>
17
18         <item name="android:textColorPrimary">@color/black</item>
19
20         <item name="android:textColorSecondary">@color/gray_99</item>
21
22         <item name="buttonBarButtonStyle">@style/DialogButtonStyle</item>
23
24         <item name="android:windowFrame">@null</item>
25
26         <item name="android:windowCornerRadius">12dp</item>
27     </style>
28
29
30     <style name="DialogTitleStyle"
parent="Base.TextAppearance.AppCompat.Title">
31         <item name="android:textSize">18sp</item>
32         <item name="android:textColor">@color/black</item>
33         <item name="android:textStyle">bold</item>
34     </style>
35
36
37     <style name="DialogButtonStyle" parent="Widget.AppCompat.Button">
38         <item name="android:textSize">16sp</item>
39         <item name="android:textColor">@color/purple_500</item>
40         <item name="android:minWidth">60dp</item>
41     </style>
42 </resources>

```

步骤 2: 创建对话框圆角背景 (res/drawable/dialog_bg.xml)

Code block

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <shape xmlns:android="http://schemas.android.com/apk/res/android"
3     android:shape="rectangle">
4     <solid android:color="@color/white"/>

```

```

5     <corners android:radius="12dp"/>
6     <stroke android:width="1dp" android:color="@color/gray_e0"/>
7 </shape>

```

步骤 3：在代码中应用自定义主题

修改 `showNativeTimePickerDialog` 方法，在 `TimePickerDialog` 构造函数中传入自定义主题 ID：

Code block

```

1 private fun showThemedTimePickerDialog() {
2     val calendar = Calendar.getInstance()
3     val currentHour = calendar.get(Calendar.HOUR_OF_DAY)
4     val currentMinute = calendar.get(Calendar.MINUTE)
5
6     // 传入自定义主题: R.style.CustomTimePickerTheme
7     val timePickerDialog = TimePickerDialog(
8         this,
9         R.style.CustomTimePickerTheme, // 应用主题
10        { _, selectedHour, selectedMinute ->
11            handleSelectedTime(selectedHour, selectedMinute)
12        },
13        currentHour,
14        currentMinute,
15        true
16    )
17
18    timePickerDialog.setTitle("选择会议时间")
19    timePickerDialog.show()
20 }

```

场景 2：完全自定义布局（替换 TimePicker 与对话框结构）

若原生 `TimePicker` 的布局（如时/分排列方式、额外功能按钮）不符合需求，可通过「自定义布局嵌入 TimePicker」实现完全定制的时间选择界面。

步骤 1：创建自定义布局（res/layout/dialog_custom_time_picker.xml）

Code block

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="wrap_content"

```

```
6      android:orientation="vertical"
7      android:padding="20dp"
8      android:background="@drawable/dialog_bg">
9
10
11      <LinearLayout
12          android:layout_width="match_parent"
13          android:layout_height="wrap_content"
14          android:orientation="horizontal"
15          android:gravity="center_vertical">
16
17          <TextView
18              android:layout_width="0dp"
19              android:layout_height="wrap_content"
20              android:layout_weight="1"
21              android:text="选择出发时间"
22              android:textSize="18sp"
23              android:textColor="@color/black"
24              android:textStyle="bold"/>
25
26          <ImageView
27              android:id="@+id/iv_close"
28              android:layout_width="24dp"
29              android:layout_height="24dp"
30              android:src="@drawable/ic_close"
31              android:clickable="true"/>
32
33      </LinearLayout>
34
35
36      <TimePicker
37          android:id="@+id/time_picker"
38          android:layout_width="match_parent"
39          android:layout_height="wrap_content"
40          android:layout_marginTop="15dp"
41          android:timePickerMode="spinner">
42      </TimePicker>
43
44
45      <LinearLayout
46          android:layout_width="match_parent"
47          android:layout_height="wrap_content"
48          android:layout_marginTop="15dp"
49          android:orientation="horizontal"
50          android:gravity="center"
51          android:spacing="10dp">
52
```

```

53         <Button
54             android:id="@+id/btn_quick_9"
55             android:layout_width="wrap_content"
56             android:layout_height="35dp"
57             android:text="9:00"
58             android:textSize="14sp"
59             android:backgroundTint="@color/gray_e0"/>
60
61         <Button
62             android:id="@+id/btn_quick_12"
63             android:layout_width="wrap_content"
64             android:layout_height="35dp"
65             android:text="12:00"
66             android:textSize="14sp"
67             android:backgroundTint="@color/gray_e0"/>
68
69         <Button
70             android:id="@+id/btn_quick_18"
71             android:layout_width="wrap_content"
72             android:layout_height="35dp"
73             android:text="18:00"
74             android:textSize="14sp"
75             android:backgroundTint="@color/gray_e0"/>
76
77     </LinearLayout>
78
79
80     <Button
81         android:id="@+id/btn_confirm"
82         android:layout_width="match_parent"
83         android:layout_height="45dp"
84         android:layout_marginTop="20dp"
85         android:text="确认选择"
86         android:textSize="16sp"
87         android:backgroundTint="@color/purple_500"/>
88
89

```

注：`ic\_close.png` 为关闭图标，需放入 `res/drawable` 目录；`timePickerMode` 为时间选择器模式，`spinner` 为滚轮模式，`clock` 为时钟模式（API 21+ 支持）。

步骤 2：在代码中加载布局并绑定逻辑

（注：文档部分内容可能由 AI 生成）